How Far Have We Gone in Binary Code Understanding Using Large Language Models

Xiuwei Shang[1], Shaoyin Cheng[1,2,†], Guoqiang Chen[1], Yanming Zhang[1], Li Hu[1], Xiao Yu[1], Gangyang Li[1], Weiming Zhang[1,2], Nenghai Yu[1,2]
{shangxw,ch3nye,azesinter,pdxbshx,yuxiao1217,ligangyang}@mail.ustc.edu.cn
{sycheng,zhangwm,ynh}@ustc.edu.cn
[1]University of Science and Technology of China, Hefei, China
[2]Anhui Province Key Laboratory of Digital Security, Hefei, China

***Abstract*—Binary code analysis plays a pivotal role in various software security applications, such as software maintenance, malware detection, software vulnerability discovery, patch analysis, etc. However, unlike source code, understanding binary code is challenging for reverse engineers due to the absence of semantic information. Therefore, automated tools are needed to assist human players in interpreting binary code. In recent years, two groups of technologies have shown promising prospects: (1) Deep learning-based technologies have demonstrated competitive results in tasks related to binary code understanding, furthermore, (2) Large Language Models (LLMs) have been extensively pre-trained at the source-code level for tasks such as code understanding and generation. This makes participants wonder about the ability of LLMs in binary code understanding.**

In this work, we propose a benchmark to evaluate the effectiveness of LLMs in real-world reverse engineering scenarios. The benchmark covers two key binary code understanding tasks, including function name recovery and binary code summarization. We gain valuable insights into their capabilities and limitations through extensive evaluations of popular LLMs using our benchmark. Our evaluations reveal that existing LLMs can understand binary code to a certain extent, thereby improving the efficiency of binary code analysis. Our results highlight the great potential of the LLMs in advancing the field of binary code understanding.

***Index Terms*—Reverse Engineering, Binary Code Understanding, Program Comprehension, Large Language Models**

I. Introduction

Binary code analysis is fundamental to software security, serving as the bedrock technology for many critical tasks including reverse engineering [1], software vulnerability detection [2], and malware analysis [3]. The process of compilation, however, leads to the elimination of semantic information present at the source-code level. Additionally, binary files often have their symbol information stripped [4] for various reasons (e.g., copyright protection [5] and obscuring functionality [6]). Thus, it is challenging for reverse engineers to understand the semantics of binary code.

Although many decompilation tools, such as IDA Pro [7], Ghidra [8] and BinaryNinja [9], can heuristically convert binary code into C-like pseudo code, they still lack easy-to-understand semantics information, especially function names and code comments that play an important role in comprehending the code [10], [11]. Recently, borrowing ideas from Natural Language Processing (NLP), deep learning-based methods have been proposed for understanding binary code. In the task of function name recovery, NERO [12], NFRE [11] and SymLM [13] utilized the disassembled assembly instruction sequence as neural models input to reassign descriptive names. NER [14] utilized decompiled pseudo code with a higher abstraction level as input and achieves better performance.

Besides, the function name alone is not enough to represent the complete behavior of the code [15]. If a natural language description can be generated for the binary code, it will greatly save the reverse engineer's analysis time. BinT5 [16] is the first generation model designed for binary code summarization, which is based on the source code model CodeT5 [17] and fine-tuned on binaries. As a unified pre-training model, HexT5 [18] can perform multiple downstream tasks such as code summarization and function name recovery. However, the methods mentioned above generally exhibit limited generalization to unseen binary code.

Recently, Large Language Models (LLMs) have attracted significant attention in the academic community. General LLMs, such as Llama [19], ChatGPT [20], etc., have been widely demonstrated for their capabilities in natural language understanding and generation. Furthermore, LLMs in the programming language domain (e.g., CodeLlama [21] and WizardCoder [22]) have shown notable ability in program analysis tasks, like fixing security vulnerabilities [23], test cases auto-generation [24]. These developments demonstrate the potential of LLMs to handle complex and structured information flows that are particularly important for understanding binary code.

In this paper, instead of developing a new technique, we investigate and compare the capabilities of various LLMs in understanding binary code. By harnessing the advanced analytical power of LLMs, we seek to explore the extent to which these models are able to understand binary code, a task that traditionally handled by skilled human engineers [25]. To facilitate our evaluation, we designed an automated approach to construct an evaluation benchmark dataset, which includes aligned source code, natural language summaries, and decompiled pseudo code. We then contrasted the capabilities

[†]Corresponding Author.

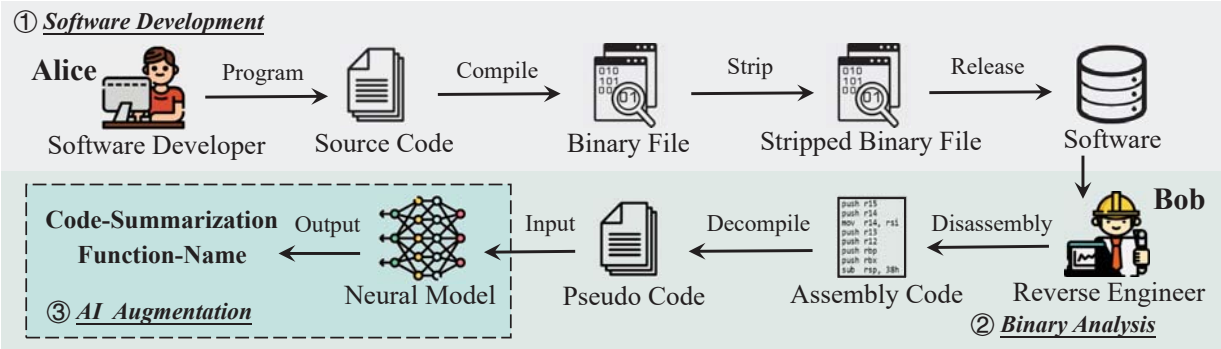

Fig. 1: Application background of binary code understanding.

of LLMs on two binary code understanding tasks, namely: (1) function name recovery, and (2) binary code summarization. We extensively evaluated eight code domain LLMs (CodeGen [26], WizardCoder [22], DeepSeek-Coder [27], CodeLlama [21] et.al.), eight general domain LLMs (ChatGLM [28], Vicuna [29], Llama [19], Mistral [30], ChatGPT [20] et.al.), and four deep learning-based expert models (SymLM [13], NER [14], BinT5 [16], HexT5 [18]). Additionally, we explored the impact of injecting domain knowledge by fine-tuning LLMs on specific tasks. Furthermore, we conducted case studies in the context of virus analysis to showcase the performance of the LLMs in understanding binary code in real-world scenarios.

Our findings demonstrate that LLMs exhibit excellent potential in advancing automated binary code understanding. We call for more research in this area to further enhance the capabilities of LLMs to play a more critical role in the complex task of binary code analysis.

Our contributions can be summarized as follows:

- We design an automated method to construct a benchmark dataset to evaluate the capabilities of binary code understanding and release it to facilitate further research.
- We conduct a thorough study that evaluates the capabilities of eight code domain LLMs, eight general domain LLMs, and four DL-based methods on binary code understanding. Our primary focus lies on two fundamental tasks: function name recovery and binary code summarization.
- Our findings provide valuable insights into the capabilities and limitations of LLMs for understanding binary code. We thoroughly discuss the outcomes of our evaluations and offer suggestions for future research directions, aiming to propel advancements in this domain.

II. Background and Motivation

A. Binary Code Understanding

As shown in Figure 1, consider a scenario where a software developer, Alice, who writes a program in a high-level language like C/C++. Her code, written in a human-readable format (Figure 2a), must be translated into a form the computer can execute. This is where the compilation process comes into play, turning her source code into binary code, a series of 1s and 0s that the machine understands.

```
/**
 *  Implement AES encryption and decryption in CBC mode.
 ↪  It takes in ...
 */
void AES_cbc_encrypt (const unsigned char *in, unsigned
↪  char *out, size_t len, const AES_KEY *key, unsigned
↪  char *ivec, const int enc) {
    if (enc)
        CRYPTO_cbc128_encrypt (in, out, len, key, ivec,
        ↪  AES_encrypt);
    else
        CRYPTO_cbc128_decrypt (in, out, len, key, ivec,
        ↪  AES_decrypt); }
```

(a) Source Code

```
0007E5F0     test     r9d, r9d
0007E5F3     jz       short loc_7E608
0007E5F5     lea      r9, AES_encrypt
0007E5FC     jmp      CRYPTO_cbc128_encrypt
0007E608     lea      r9, AES_decrypt
0007E60F     jmp      CRYPTO_cbc128_decrypt
```

(b) Assembly Code

```
0007E5F0     test     r9d, r9d
0007E5F3     jz       short loc_7E608
0007E5F5     lea      r9, sub_0BA6E0
0007E5FC     jmp      sub_1B6E10
0007E608     lea      r9, sub_0BAAA0
0007E60F     jmp      sub_1B6F50
```

(c) Stripped Assembly Code

```
_int64 _fastcall sub_0BA040 (_int64 a1, _int64 a2,
↪  _int64 a3, _int64 a4, _int64 a5,int a6) {
    __int64 result; // rax
    if ( a6 )
        result = sub_1B6E10 (a1, a2, a3, a4, a5,
        ↪  sub_0BA6E0);
    else
        result = sub_1B6F50 (a1, a2, a3, a4, a5,
        ↪  sub_0BAAA0);
    return result; }
```

(d) Stripped Pseudo Code

Fig. 2: Example of source, assembly, stripped assembly and stripped pseudo code snippet.

Once compiled, Bob, a reverse engineer, wants to understand how Alice's program works. He uses a disassembler to convert the binary code back into an assembly code sequence (Figure 2b), which is more readable than binary but still quite low-level. However, Alice has used the "strip" command to remove symbol information from the original binary code in order to

reduce file size and protect intellectual property. This makes Bob's job more difficult because he is now missing critical information such as function names (replaced with meaningless placeholders such as `sub_0BA6E0`) and code comments (Figure 2c).

Finally, Bob uses a decompiler in an attempt to reconstruct the high-level logic of the program. The decompiler generates pseudo code (Figure 2d), an approximation of what Alice's original source code might have looked like. However, due to the complexity of the decompilation process and the lack of symbolic information in the stripped binary, the pseudo code may not exactly match Alice's original code, making it still difficult for Bob to understand the function of the program.

At this point, Bob attempts to use advanced natural language processing (NLP) techniques, such as LLMs or deep learning models, which are adept at identifying patterns and inferring logical structures. Bob leverages these techniques to **predict function names** and **generate natural language summaries** of the code's functionality. This process can be formalized as:

$$\mathcal{N},\ \mathcal{S} = f(\mathcal{F}, \mathcal{B}) \tag{1}$$

where $\mathcal{F}$ is a stripped decompiled function in pseudo code form in binary file $\mathcal{B}$, which is fed into LLMs denoted as f. The objective is to generate a meaningful function name denoted as $\mathcal{N}$, and a natural language description denoted as $\mathcal{S}$ of this function.

Through this process, Bob combined the analytical power of AI with his reverse engineering skills to bridge the gaps left by the stripped binaries and gain a deeper understanding of Alice's original programming intent.

B. Related Works

1) Function Name Recovery: This task aims to predict and recover descriptive function names for functions in binary files without symbolic information, which is of great significance for summarizing function semantics and understanding software behavior. Several methods based on deep learning have been proposed. Among them, NERO [12] uses augmented control flow graphs, combined with the neural model of the encoder-decoder paradigm, to effectively capture the behavioral characteristics of functions and provide a new method for recovering binary function names. NFRE [11] proposes a lightweight framework for function name recovery that utilizes the sequential and structural information of assembly instructions. The efficiency and scalability of the framework provide the possibility to process a large-scale binary files. Based on NFRE, SymLM [13] further considers the calling context to help the model understand function semantics, and leverages advanced pre-training models [31] for instruction embedding. Finally, NER [14] started from the perspective of binary code representation and studied the effectiveness of different representations for function name recovery using deep neural models, providing new perspectives and tools for this field.

2) Binary Code Summarization: Code summarization aims to extract key information from binary code and generate concise summaries, which is important for supporting program understanding. BinT5 [16] is the first model focused on binary code summarization, which extends the application scope of source code pre-trained language models. This model treats the decompiled code as a special programming language, uses fine-tuned CodeT5 [17] to capture its semantics and generate a summary. The introduction of BinT5 opens up new avenues for binary code summarization research. Furthermore, HexT5 [18] proposed a unified pre-training model also based on CodeT5, which allowed multi-task learning, supported function name recovery, binary code summarization and other downstream tasks, and demonstrated promising performance.

Project	Domain	# Binaries	# Functions	# Select
FFmpeg [34]	Video	2	30,585	200
Redis [35]	Database	2	6,837	200
Curl [36]	Network	2	6,688	200
Masscan [37]	Network	1	536	150
Llama2.c [38]	Neural Network	2	35	34
Whisper.cpp [39]	Neural Network	9	5,339	200
OpenSSL [40]	Crypto	2	19,122	300
zstd [41]	Compress	1	2,432	200
ImageMagick [42]	Image	3	3,798	200
Libvips [43]	Image	1	5,424	200
Libexpat [44]	Format	1	362	100
Ultrajson [45]	Format	2	27	16
Total (12)	8	28	81,185	2,000

TABLE I: Statistics of our benchmark dataset.

C. Our Motivation

Large Language Models (LLMs), often composed of billions or even trillions of parameters, are built upon large amounts of data to learn the relationship between program language and natural language. Recently, A few studies [23], [24], [32] find that LLMs have demonstrated excellent capabilities in dealing with natural language tasks, as well as source code understanding, indicating that they have the potential to be applied to complex analysis of source code. Given this, we speculate that these models may be equally applicable to binary code understanding. Since binary code is similar in nature to source code and natural language, they all follow certain patterns and structures that can be learned and exploited by LLMs [33].

Therefore, this study will explore the potential of LLMs in understanding binary code, aiming to evaluate whether these models can cross domain boundaries and extend their capabilities in natural language and source code to binary code analysis. This is expected not only to provide new perspectives for automated code understanding, but also to open up new application paths in areas such as reverse engineering and malware analysis.

III. Evaluation Design

A. Dataset Construction

Before we can effectively evaluate the ability of LLMs to understand binary code, a comprehensive benchmark dataset is

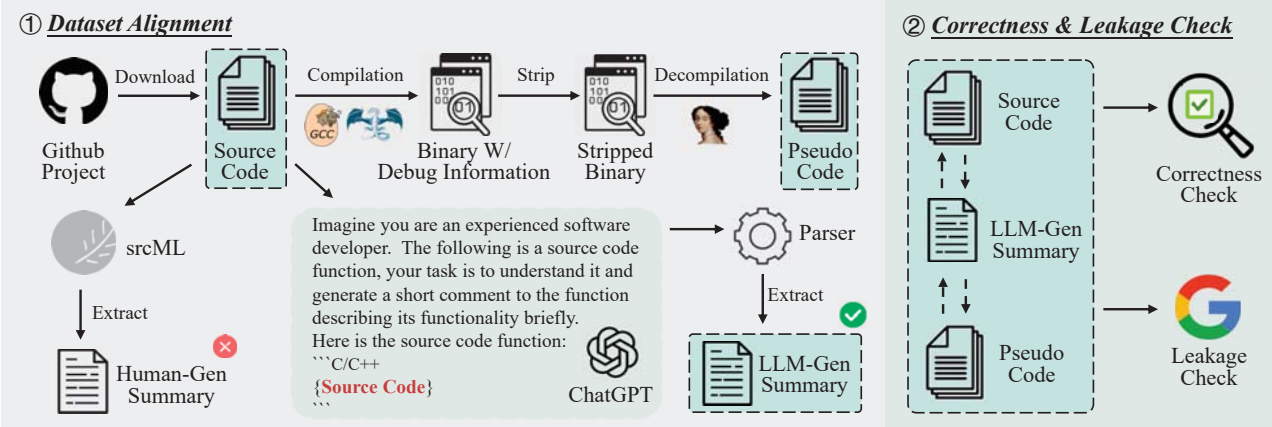

Fig. 3: An overview of the benchmark dataset construction process.

necessary to provide a consistent basis for different model evaluations and comparisons. The specific process of constructing the benchmark dataset is shown in Figure 3.

*1) **Source code selection**:* To reflect real-world reverse engineering needs, we believe that code sources of the benchmark should meet the following requirements:

- *Authenticity*: the code should come from real projects, not toy programs or incomplete code snippets.
- *Breadth*: the selected code should cover multiple fields and not be limited to specific fields or application scenarios.
- *High quality*: the selected code should be of good quality, including clear structure, reasonable naming conventions, etc.
- *Credibility*: the selected code should ideally be sourced from projects maintained by well-known or reputable developers or organizations to accurately reflect real-world application scenarios.

Therefore, as shown in Table I, we select 12 real-world projects implemented in C language with the highest star ratings on Github, which have high credibility, excellent code quality and maintenance standards, covering eight application domains, including crypto, compress, network, video, image, database, neural network, etc.

*2) **Compile, strip and decompile**:* We compile these projects on a machine running Ubuntu 22.04 OS, targeting the *x86_64* architecture, and for compiler and optimization level we use the default configuration of each project. As illustrated in Table I, we generate a total of 28 binaries. Subsequently, we employ the `strip` command in Linux to remove the symbol tables from these binaries.

Previous research [14] has found that using pseudo code as a representation of binary code is more effective for neural models than assembly instruction sequence and Intermediate Representation (IR). Therefore, we directly utilize IDA Pro [7] to decompile the binary files and convert the binary code into pseudo code form without considering other representation forms.

*3) **Alignment**:* We use DWARF [46] debugging information to align source code and pseudo code, which can record functions, variables in binary functions, and their locations (include source file name, line number, and column number) in source code. As shown in Table I, we obtain a total of 81,185 functions matching source code and pseudo code. To align source code and human-written summary, we use srcML [47] to analyze and parse the source files, then collect single- and multi-line summaries above the location of function declarations and definitions. This completes the alignment of source code – pseudo code – human summary.

*4) **Ground-truth identifiction**:* For the function name recovery task, we parse the function names in the source code as labels using regular expressions. For the binary code summarization task, we first consider using human-written comments extracted from source code files as labels as in previous work [16], [18]. However, we found that only about 14.8% of functions have comments written by human developers. Worse yet, not all comments are describing the functional summary of the function, but will also contain some noisy content, and they are of varying quality and style. Therefore, using human-written comments as ground-truth is unreliable.

Presently, an increasing number of research works [48]–[50] employ large language models such as ChatGPT [20] for tasks like data annotation, and has demonstrated a certain degree of reliability. Inspired by these pioneering works, we utilize ChatGPT to generate summaries as ground-truth. Specifically, we use the source code of the function to construct the prompt shown in Figure 3, prompting ChatGPT to generate a short summary describing the function's purpose and functionality.

*5) **Correctness & leakage check**:* It is crucial to ensure the correctness of the ground-truth, so we perform a correctness check on the descriptive summaries generated by ChatGPT. Specifically, we invited three experienced domain experts to review the match between the source code and the summary. Experts were asked to give each abstract a "pass" or "fail" score. If two or more experts give a "fail" rating, the data will be removed directly from the dataset; if one expert gives a "fail" rating, we will conduct a collective discussion and give a final in conclusion. We finally select 2,000 pieces of data as

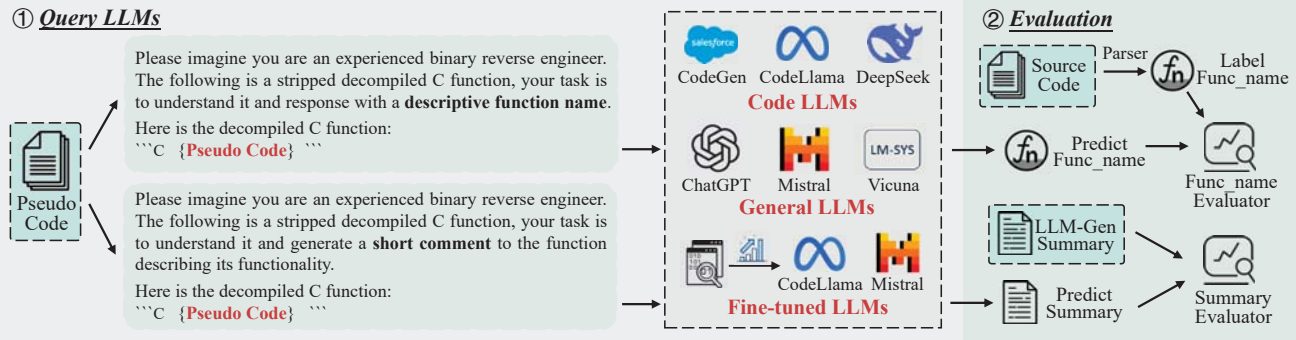

Fig. 4: An overview of the evaluation process.

Domain	Model	Release Time	Size	Base Model	Training Corpus			Publisher	License
					Raw Size	#Tokens	#Instances		
Code LLMs	CodeGen25-7b-instruct [26]	Jul-2023	7B	CodeGen2	-	1.4T	-	Salesforce	Open-source
	WizardCoder-15b-V1.0 [22]	Jun-2023	15B	StarCoder	-	-	78.0K	WizardLM	Open-source
	WizardCoder-33b-V1.1 [22]	Jan-2024	33B	Deepseek-Coder	-	-	-	WizardLM	Open-source
	Code Llama-7b-instruct-hf [21]	Jun-2023	7B	Llama-2-7b	4.4TB	525.0B	-	Meta AI	Open-source
	Code Llama-13b-instruct-hf [21]	Jun-2023	13B	Llama-2-13b	4.4TB	525.0B	-	Meta AI	Open-source
	Code Llama-34b-instruct-hf [21]	Jun-2023	34B	Llama-2-34b	4.4TB	525.0B	-	Meta AI	Open-source
	Code Llama-70b-instruct-hf [21]	Jan-2024	70B	Llama-2-70b	-	1.0T	-	Meta AI	Open-source
	DeepSeek-Coder-33b-instruct [27]	Nov-2023	33B	-	-	2.0T	-	DeepSeek-AI	Open-source
General LLMs	ChatGLM2-6B [28]	Jun-2023	6B	-	-	1.4T	-	THUDM	Open-source
	Vicuna-7b-v1.5 [29]	Aug-2023	7B	Llama-2-7b	-	-	125.0K	L.Zheng et al.	Open-source
	Vicuna-13b-v1.5 [29]	Aug-2023	13B	Llama-2-13b	-	-	125.0K	L.Zheng et al.	Open-source
	Llama-2-13b-chat-hf [19]	Jul-2023	13B	-	-	2.0T	-	Meta AI	Open-source
	Llama-2-70b-chat-hf [19]	Jul-2023	70B	-	-	2.0T	-	Meta AI	Open-source
	Mistral-7B-Instruct-v0.2 [30]	Dec-2023	7B	Mistral-7B	-	-	-	Mistral AI	Open-source
	Mixtral-8x7B-Instruct-v0.1 [30]	Dec-2023	47B	Mistral-7B	-	-	-	Mistral AI	Open-source
	ChatGPT [20]	Nov-2022	-	-	-	-	-	OpenAI	Closed-source

TABLE II: Detail information of Large Language Models we apply in this work.

the benchmark dataset.

It is also imperative that benchmark datasets are not included in the training set of LLMs to mitigate the risk of data leakage. All our evaluation data are decompiled pseudo code, and the symbolic information is stripped away so that it is significantly different from the corresponding source code form, which greatly avoids data leakage. To further ensure the validity and reliability of our benchmark evaluation, we use the Google search engine to check whether the code appears on the Internet in clear text. The results showed that none of the pseudo codes were retrieved by whole-word matching.

B. Large Language Models Setup

*1) **Large Language Models As Is**:* We select eight code domain LLMs, i.e., CodeGen25 [26], DeepSeek-Coder [27], two versions of WizardCoder [22], four versions of CodeLlama [21], and select eight general domain LLMs, i.e., ChatGLM [28], two versions of Vicuna [29], two versions of Llama [19], two versions of Mistral [30], and ChatGPT [20]. The principles for our selection are: (1) state-of-the-art, (2) pre-trained on enough source code to be able to understand code to a certain extent, and (3) have text generation and code generation capabilities. In addition, in order to ensure that the model can follow the instructions, we all choose the instruct-tuned version. Table II provides detailed information, including parameter size, base model, training corpus, publisher, etc.

Limited by the context window length, we set the maximum input tokens to 4,096, and code snippets exceeding the length will be truncated. For the function name recovery task, we set the maximum new tokens to 48, and for the code summarization task, we set it to 256. We set the sampling temperature to 1, top_p to 0.95, top_k to 10, and num_beams to 1. For all open-source models, we downloaded them from HuggingFace [51] and deployed on our local machine with FP16 mixed precision enabled during inference. For ChatGPT, we called its latest `gpt-3.5-turbo-16k` version through the OpenAI's API.

*2) **Prompt Formats**:* Figure 4 illustrates the prompt format we used for LLMs. We use role-play [52], [53] prompts to give LLMs the role of experienced binary reverse engineers, enabling them to better handle binary code understanding tasks. Considering the limitation of the length of the model context window, and in order to reduce the inference time overhead and memory usage, we adopt the zero-shot prompts. We analyze the impact of few-shot prompts on the performance of LLMs in Section IV-C.

*3) **Fine-tuned Large Language Models**:* We also investigate the ability of fine-tuned LLMs to understand binary code, since fine-tuning is a common technique to adapt a pre-trained LLM to downstream tasks [23], [54], [55], such as function name recovery and code summary generation. Furthermore, pre-training corpora of existing LLMs contain very few binary code, either in the form of disassembled instruction sequences

or decompiled pseudo code. Therefore, we hope to explore whether injecting binary domain information into LLMs can improve its performance.

The GNU repository[1] is extensively used as a training or test set for many existing deep learning-based works [12]–[14], [18], [56], [57]. To build our fine-tuning dataset, we select 51 projects from the GNU repository, including binutils, coreutils, findutils, libmicrohttpd, nettle, etc. We use BinKit [58] to create a compilation environment, and then compiler the selected projects using the *gcc-11.2.0* compiler with *x86_64* target architecture and *O0* optimization level. We obtained a total of 270 binary files. After strip, decompile and alignment, we obtained 124,819 functions matching source code and decompiled pseudo code, and randomly selected 30,000 of them as the fine-tuning dataset.

Additionally, we perform a search and confirm that none of the functions in our proposed benchmark is present in the fine-tuning dataset.

C. Evaluation Setup

The evaluation environment is a machine equipped with 8 * NVIDIA RTX A6000 GPU with 48GB of VRAM, 2 * 28-core Intel Xeon 6330 CPU, 512GB RAM and 64TB storage, running on Ubuntu 22.04 OS. The GPU is running Nvidia driver version 525.116.03 with CUDA version 12.0.

We implement all the experiments using Python 3.8 with PyTorch [59] 2.0.1, DeepSpeed [60] 0.13.0 and Transformers [61] 4.37.2 packages. As for model fine-tuning, we implement it based on the LLaMa-Factory [62] framework. We use LoRA [63] fine-tuning method and specify all available modules. We adopt Adam optimizer in fp16 precision, 40 global batch size and 1 training epoch. The learning rate is set to 1e-5 and followed by a consine decay.

IV. Evaluation Results and Findings

In this section, we conduct extensive experiments to answer the following research questions:

- **RQ1:** How do LLMs perform in the task of function name recovery?
- **RQ2:** How do LLMs perform in the task of binary code summarization?
- **RQ3:** What factors significantly influence the performance of LLMs to understand binary code?
- **RQ4:** Can fine-tuning enhance the capability of LLMs to understand binary code?
- **RQ5:** Do LLMs have the practical ability in real-world scenarios?

A. RQ1: Performance of Function Name Recovery

Metric. For the function name recovery task, following [11], [14], we calculate token-level Precision, Recall, F1-score to evaluate the performance of LLMs. The metric ignore non-alphabetical characters and are case-, order-, and duplication-insensitive at the token-level. They can be expressed as:

[1]http://ftp.gnu.org/gnu

TABLE III: Comparison of LLMs and DL-based methods on function name recovery. We mark the best performing methods in each domain.

Domain	Model	Metric			
		Precision	*Recall*	*F1-score*	*Time(s)*
Code LLMs	CodeGen25-7b-instruct	13.57	15.09	13.50	1.47
	WizardCoder-15b-V1.0	26.41	26.14	25.40	2.22
	WizardCoder-33b-V1.1	21.89	22.48	21.19	5.09
	Code Llama-7b-instruct-hf	28.31	27.21	26.87	0.74
	Code Llama-13b-instruct-hf	28.26	27.30	26.76	1.60
	Code Llama-34b-instruct-hf	28.96	28.44	27.59	3.69
	Code Llama-70b-instruct-hf	26.92	27.42	26.19	9.76
	DeepSeek-Coder-33b-instruct	25.30	27.32	25.22	4.11
General LLMs	ChatGLM2-6B	13.98	16.36	14.12	0.59
	Vicuna-7b-v1.5	22.35	21.36	21.16	0.61
	Vicuna-13b-v1.5	25.26	24.89	24.34	0.96
	Llama-2-13b-chat-hf	26.78	28.31	26.69	1.19
	Llama-2-70b-chat-hf	27.45	27.14	26.68	4.68
	Mistral-7B-Instruct-v0.2	23.38	27.49	24.28	0.84
	Mixtral-8x7B-Instruct-v0.1	19.77	25.62	21.26	2.13
	ChatGPT(gpt-3.5-turbo-16k)	22.85	22.26	21.85	1.43
DL-based	SymLM [13]	10.69	6.04	7.72	0.29
	NER [14]	17.10	12.03	13.33	0.03
	HexT5 [18]	3.69	3.33	3.28	0.17

$$\text{Precision} = \frac{\text{TP}}{\text{TP + FP}} \tag{2}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP + FN}} \tag{3}$$

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

Result. Table III shows the performance of LLMs and DL-based methods on the function name recovery task.

We observe outstanding performance for each parameter size version of CodeLlama and Llama. Among them, CodeLlama-34b outperforms all other LLMs in precision, recall and F1-score metrics, obtaining scores of 28.96, 28.44 and 27.59 respectively. Following closely behind, WizardCoder-15b and DeepSeek-Coder-33b also show excellent performance, obtaining F1-score of 25.40 and 25.22 respectively. CodeGen25-7b and ChatGLM2-6B perform the worst, with only 48.9% to 66.7% of other LLMs on the F1-score metric. This lagging performance may be attributed to the capability flaws of their basic models or the lack of targeted training data.

Furthermore, as a task that combines natural language and code language understanding, code domain LLMs generally perform slightly better than general domain LLMs. This may be because their pre-training datasets have a higher proportion of source code. Training on a extensive range of source code datasets allows them to gain a deeper grasp of programming syntax, structure and semantics.

> **Findings 1:** CodeLLama-34b performs best in the function name recovery task, with an F1-score of 27.59%. Code domain LLMs generally perform slightly better than general domain LLMs, likely owing to their greater

> familiarity with programming paradigms.

Among the deep learning-based expert models, NER [14] performs best, achieving an F1-score of 13.33, but it is still slightly behind CodeGen25, the worst-performing LLM. However, SymLM [13] and HexT5 [18] only obtain F1-scores of 7.72 and 3.28, respectively, which differs from the performance reported in their original articles. This difference may come from the partitioning of their datasets. SymLM and HexT5 widely use projects from GNU as part of their training and testing sets. SymLM divides the training and testing sets at the binary file-level, which may result in some code appearing in both the training and testing sets. For example, in the `Binutils` project, the `ar` and `nm` files share the same binary file descriptor(BFD) processing code. This reuse of libraries and underlying code may lead to exaggeration of evaluation metrics. Although HexT5 adopts a stricter project-level dataset partitioning approach, different projects under GNU may still share programming styles or naming conventions, leading to potential data leaks. In summary, these models have limited generalization ability for data outside the training data distribution, and this zero-shot learning ability is exactly what LLMs are good at.

> **Findings 2:** The existing DL-based expert models have poor generalization ability for data outside the training distribution, and their performance is far lower than the LLMs.

In terms of inference time cost, locally deployed LLMs with 6B-7B parameter quantities typically require 0.6 to 0.8 seconds to infer a single piece of data, LLMs with 13-15B scales require 1.2 to 2.2 seconds, LLMs with 33-34B scales require 4.1 to 5.1 seconds, and CodeLlama-70b requires a maximum of 9.76 seconds per piece. For ChatGPT that requires API calls, inference for each data takes 1.43 seconds. The DL-based model, due to its lightweight advantage, greatly reduces inference time and achieves the fastest NER of 0.03 seconds per piece.

> **Findings 3:** The DL-based model has a significant advantage in inference speed benefiting from their model size. Meanwhile, the inference speed of LLMs is still within an acceptable range.

B. RQ2: Performance of Binary Code Summarization

Metric. For the binary code summarization task, same as BinT5 [16], HexT5 [18], we use smoothed BLEU-4 [64], METEOR [65], Rouge-L [66] as the evaluation metric.

Result. The performance of LLMs and DL-based methods on the binary code summarization task is presented in Table IV.

ChatGPT outperforms all other LLMs in BLEU-4, METEOR and Rouge-L metrics, obtaining scores of 7.37, 28.13 and 23.80 respectively. WizardCoder-15b also shows very competitive results, achieving 6.60, 26.53, and 23.80, respectively. Similar to the function name recovery task, CodeGen25-7b and ChatGLM2-6B performed the worst in their respective domains, but narrowed the performance gap with other LLMs.

TABLE IV: Comparison of LLMs and DL-based methods on binary code summarization. We mark the best performing methods in each domain.

Domain	Model	Metric			
		BLEU-4	*METEOR*	*Rouge-L*	*Time(s)*
Code LLMs	CodeGen25-7b-instruct	3.87	23.61	18.95	7.32
	WizardCoder-15b-V1.0	6.60	26.53	23.80	12.91
	WizardCoder-33b-V1.1	4.74	25.43	20.34	23.71
	Code Llama-7b-instruct-hf	4.65	22.20	20.66	5.33
	Code Llama-13b-instruct-hf	4.03	19.43	19.68	8.37
	Code Llama-34b-instruct-hf	4.69	21.13	21.14	18.93
	Code Llama-70b-instruct-hf	4.64	23.75	20.82	53.55
	DeepSeek-Coder-33b-instruct	5.22	24.10	20.84	20.26
General LLMs	ChatGLM2-6B	4.29	26.37	20.66	5.53
	Vicuna-7b-v1.5	5.99	25.29	22.84	5.54
	Vicuna-13b-v1.5	4.90	21.20	22.48	7.53
	Llama-2-13b-chat-hf	6.16	24.47	22.51	11.30
	Llama-2-70b-chat-hf	5.51	26.26	21.51	47.76
	Mistral-7B-Instruct-v0.2	5.98	24.37	23.55	3.64
	Mixtral-8x7B-Instruct-v0.1	5.79	24.16	23.56	11.00
	ChatGPT(gpt-3.5-turbo-16k)	7.37	28.13	23.76	2.85
DL-based	BinT5 [16]	0.00	2.12	4.83	0.59
	HexT5 [18]	0.09	6.32	8.53	0.47

Unlike the function name recovery task, the performance of general domain LLMs is generally significantly better than that of code domain LLMs in binary code summary tasks. This may be attributed to the different properties of the two tasks. In the function name recovery task, the output of LLMs is usually shorter and only needs to generate a function name, which is relatively simple. In contrast, the binary code summarization task requires generating longer natural language descriptions to accurately summarize the functionality and structure of binary code, which requires the model to understand more contextual information and convert it into natural language text, which is a more complex task. General domain LLMs are better at generating longer natural language descriptions due to their inherent characteristics, while code domian LLMs have limited capabilities in this regard.

> **Findings 4:** ChatGPT performs best among all LLMs in the binary code summarization task with a BLEU-4 of 7.37%. General domain LLMs perform significantly better than code domain LLMs, which is attributed to its stronger long-context understanding and summarizing capabilities.

For the DL-based expert model, BinT5 [16] obtains 0.00, 2.12 and 4.83 on the BLEU-4, METEOR, Rouge-L metric, and HexT5 [18] improves slightly, obtaining 0.09, 6.32 and 8.53 respectively, but the performance is still far lower than LLMs.

> **Findings 5:** Similar to the previous task, existing DL-based expert models perform worse than LLMs on the

TABLE V: Performance of prompts in the form of Few-shot. The Impr. column represents the performance improvement of Few-shot compared to Zero-shot. We mark the increase and decrease of the metrics.

Model	Function Name Recovery								Binary Code Summarization							
	Precision		*Recall*		*F1-score*		*Time(s)*		*BLEU-4*		*METEOR*		*Rouge-L*		*Time(s)*	
	Few	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**	**Few**	**Impr.**
WizardCoder-15b-V1.0	30.65	+4.24pt	30.90	+4.76pt	29.17	+3.77pt	1.82	-0.40s	7.30	+0.70pt	29.69	+3.16pt	24.52	+0.72pt	11.32	-1.59s
Code Llama-7b-instruct-hf	29.64	+1.33pt	27.86	+0.65pt	28.01	+1.14pt	1.13	+0.39s	6.62	+1.97pt	27.64	+5.44pt	23.58	+2.92pt	6.69	+1.36s
Vicuna-7b-v1.5	27.24	+4.89pt	25.44	+4.08pt	25.52	+4.36pt	0.88	+0.17s	6.68	+0.69pt	22.39	-2.90pt	21.35	-1.49pt	4.43	-1.11s
Mistral-7B-Instruct-v0.2	31.45	+8.07pt	31.87	+4.38pt	30.73	+6.45pt	1.06	+0.22s	7.94	+1.96pt	29.32	+4.95pt	25.00	+1.45pt	5.96	+2.32s
ChatGPT(gpt-3.5-turbo-16k)	32.96	+10.11pt	31.17	+8.91pt	31.24	+9.39pt	1.79	+0.36s	9.71	+2.34pt	32.13	+4.00pt	27.21	+3.45pt	3.50	+0.65s

binary code summarization task.

Regarding inference time, locally deployed LLMs are generally 5-6 times longer than the function name recovery task. LLMs with 6B-7B parameters usually take 3.6 to 7.3 seconds to infer a single piece of data, 13-15B scale LLMs take 7.5 to 12.9 seconds, and 33-34B scale LLMs takes 18.9 to 23.7 seconds. CodeLlama-70b takes the most of 53.55 seconds among LLMs. For ChatGPT, which requires API calls, inference takes 2.85 seconds per data, roughly twice as long as the function name recovery task. The DL-based model also shows the advantage of inference speed, with HexT5 taking only the shortest 0.47 seconds.

Findings 6: For the binary code summarization task, inference time for locally deployed LLMs is about five times that of function name recovery.

C. RQ3: Factors that Significantly Affect Performance

We further explore the key factors affecting the performance of LLMs, including the few-shot form prompts, the length of pseudo code, and the length of symbol information.

*1) **Few-shot prompts**:* The pre-training datasets for LLMs contain little or no binary code, which makes directly applying LLMs to binary code understanding tasks likely not to yield optimal results. In this case, few-shot prompts become a potential solution, by providing well-designed examples to LLMs, so that LLMs can learn the unique structure and syntax of binary code and quickly adapt to new tasks. Specifically, we construct two carefully designed pairs of pseudo code and ground-truth examples, and add them to the original prompts. We select five LLMs that performed well in the previous experiments to conduct the few-shot prompts experiment. The results are shown in Table V.

For the function name recovery task, compared with the zero-shot prompts, general domain LLMs have a more obvious improvement compared to code domain LLMs. Among them, ChatGPT has improved by 10.11, 8.91, and 9.39 points in Precision, Recall and F1-score metric respectively. For the binary code summarization task, the LLMs of both domains have improved slightly, among which ChatGPT has improved by 2.34, 4.00, and 3.45 points in the BLEU-4, METEOR, and Rouge-L metric respectively. The exception is that Vicuna-7b shows a slight decrease in the METEOR and Rough-L metrics. Observing its output, we found that the introduction of the few-shot examples increased the length of prompts, causing more test data to exceed the maximum length of window context of the model (4096 tokens) and be truncated, resulting in a decrease in performance.

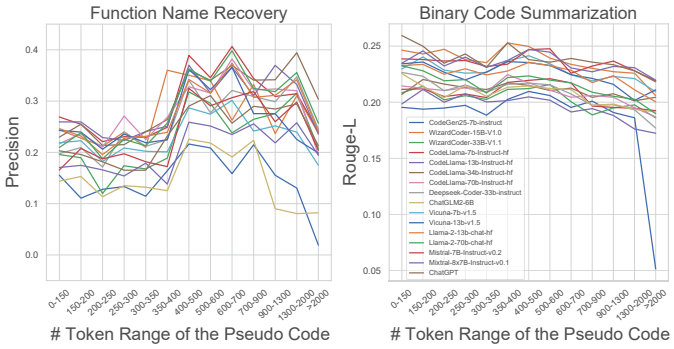

Fig. 5: Impact of pseudo code length on performance.

In addition, few-shot prompts will improve inference time in most cases. However, observing the outputs of WizardCoder-15b, we found that the few-shot prompts improve the model's ability to follow instructions, reduce the output of useless information, and thus reduce the inference time. In this case, few-shot prompts not only enhance model performance but also improve inference efficiency.

Findings 7: When computing resources and inference time permit, few-shot prompts can be selected to improve the performance of LLMs on function name recovery and binary code summarization tasks.

*2) **Pseudo Code Length**:* To study the impact of pseudo code length on the performance of LLMs, we divide the length of pseudo code token according to intervals, and controll the number of data in each interval between 100 and 200 to avoid long-tail distribution of data.

As shown in Figure 5, when the length of pseudo code is between 0-400 tokens, the metric of function name recovery remains at a relatively low level, as shorter pseudo code may not provide enough keywords to infer the purpose and naming intention of the function. Longer pseudo code can provide more

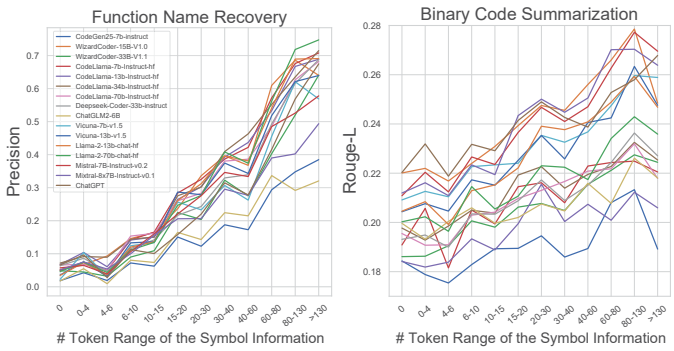

Fig. 6: Impact of symbol information length on performance.

contextual information, helping the LLMs capture semantic clues related to function names. Therefore, the metrics are relatively high between 400-2000 tokens; After exceeding 2000 tokens, the structure and logic of the code are too complex, making it difficult for the LLMs to process and integrate a large amount of information, resulting in a decrease in metrics.

For the binary code summarization task, the metrics show a slowly decreasing trend as the pseudo code length increases. As code complexity increases, LLMs find it difficult to maintain both conciseness and accuracy of summaries resulting in the generation of lengthy and unfocused summaries, thereby reducing the overall quality of the summaries.

> **Findings 8:** LLMs achieve the best performance for function name recovery at moderate pseudo code length, while the performance of binary code summarization slowly decreases as pseudo code length increases.

*3) **Symbol Information Length**:* We define symbol information as the strings and identifiers in the pseudo code that are not stripped during the strip process, which can provide human-understandable semantic information. We also divide the length of the symbolic information token into intervals.

As shown in Figure 6, as the length of the symbol information token increases, the performance of both function name recovery and binary code summarization tasks increases significantly. This is due to the fact that longer symbol information provides richer semantic content and more context clues, helping LLMs understand the intent and functionality of the code. However, we found that when the symbol information token exceeds 130, the performance of most LLMs in binary code summarization tasks slightly decreases. This is because more symbol information tokens are accompanied by longer pseudo code lengths, resulting in more code being truncated due to window context limitations, affecting the completeness of LLMs summary.

> **Findings 9:** The symbol information (e.g., strings and identifiers) has rich semantics and contributes significantly to LLM's understanding of binary code.

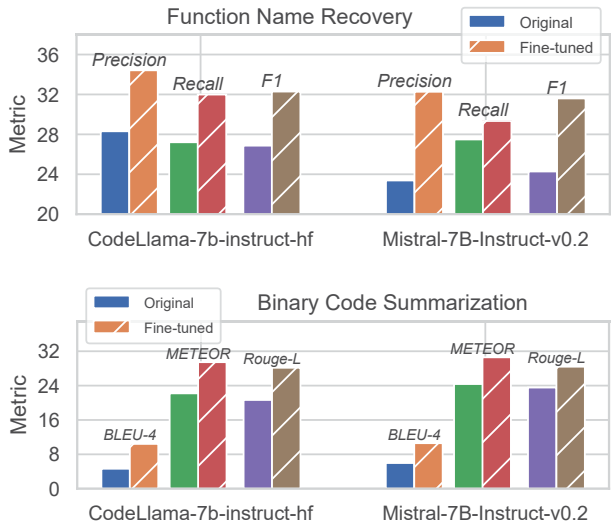

Fig. 7: Comparison of original and fine-tuned LLMs on performance.

D. RQ4: Fine-Tuning to Enhance the Performance

As mentioned in Section III-B3, we build the fine-tuning dataset from the GNU repository, with each piece of data in the form of decompiled pseudo code and ground-truth pairs. Considering the computational resource limitations, we only fine-tune the 7b LLMs, and we choose `Codellama-7b-instruct-hf` of the code domain and `Mistral-7B-Instruct-v0.2` of the general domain that have performed well in previous experiments.

Figure 7 shows the performance comparison of original and fine-tuned LLMs. For the function name recovery task, the Precision, Recall and F1-score metrics increase by an average of 7.50, 3.32 and 6.37 points respectively. For the binary code summarization task, the BLEU-4, METEOR and Rouge-L metrics increase by an average of 5.20, 6.74 and 6.18 points respectively. Overall, fine-tuning LLMs on downstream tasks related to binary code understanding can bring considerable performance improvements.

> **Findings 10:** Introducing binary domain knowledge through fine-tuning can improve the performance of LLMs on function name recovery and summary production tasks.

E. RQ5: Case Study on Real-World Virus Analysis

We present a case study to show how much advanced general LLMs can assist participants in a real-world scenario. Specifically, we utilize ChatGPT [20] to facilitate virus analysis, including summarizing the functionality of decompiled binary functions in viruses and recovering their descriptive names.

An open-source Linux remote access trojan named `splinter`[2] is compiled with *gcc-11.4.0* and stripped to release. In this case, Figure 8 has shown a partial analysis in a call

[2]https://github.com/tuian/splinter

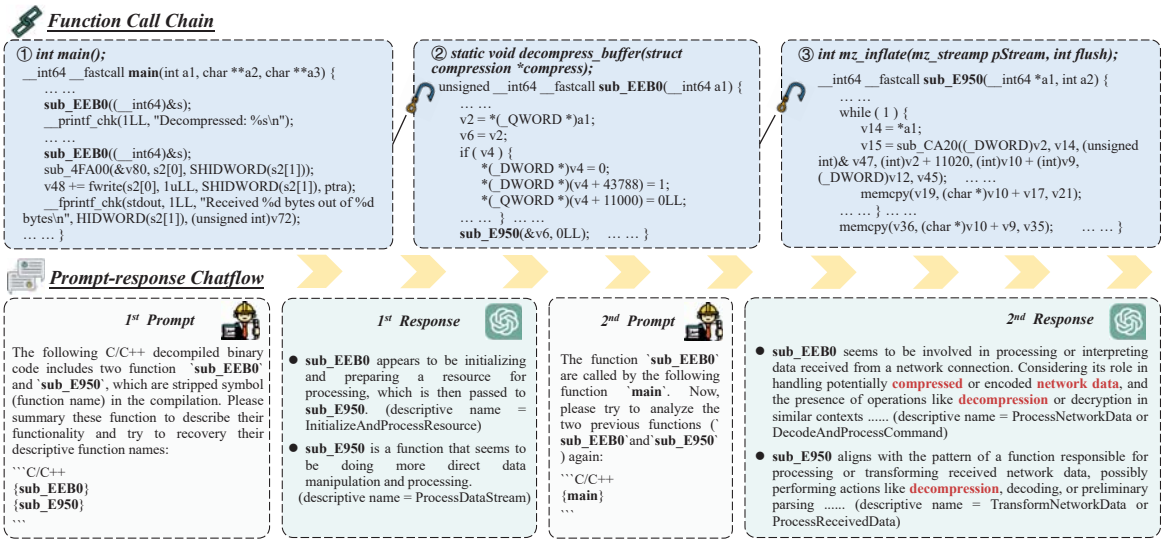

Fig. 8: An example of binary code understanding in a real-world virus with ChatGPT.

chain, where the reduced pseudo code are shown in the upper part, as well as the function definitions in the source code. The lack of symbolic information makes it difficult for an analyst to understand the function `sub_EEB0` and `sub_E950`. We first constructed the first prompt with these two functions and fed it to ChatGPT, asking for the functional summaries and descriptive names. The model gives us a primary description of the operations performed in the pseudo code without any high-level insight. We then constructed the second prompt with the caller function `main` that contains a few symbol information. The second response correctly indicates that these functions are related to data decompression. At the same time, the predicted names reflect their functionality, although the predictions do not exactly match the source code.

> **Findings 11:** ChatGPT demonstrates the potential ability to analyze binaries in the real world. The information from the calling context will boost the predictions of LLMs.

V. Discussions

Based on the experimental findings, we summarize directions for future work and limitations of the current evaluation.

A. Future Works

Current LLMs have indeed shown potential in understanding binary codes, and we believe that future work can be conducted in-depth from the following aspects.

- *Develop domain-specific LLM*: craft and refine LLMs tailored for binary code analysis and understanding, incorporating extensive binary domain knowledge during the pre-training phase to enhance the LLMs's grasp of code semantics and structure.
- *Extend context window*: investigate advanced architectures and techniques that support longer sequence lengths, allowing LLMs to effectively analyze and comprehend extensive code snippets.
- *Enhance processing of non-intuitive code*: address the current shortfall of LLMs in handling code devoid of descriptive strings or identifiers by devising innovative algorithms that can decipher and interpret the functionality of complex, non-obvious code snippets.
- *Integrate Multi-Modal Information*: explore strategies for integrating diverse information sources, such as expert human annotations, assembly instructions, and dynamic execution data, into the LLMs' input to provide a comprehensive understanding of binary code.

B. Limitations

Although this paper provides a systematic evaluation of the performance of LLMs on binary code understanding tasks, we need to acknowledge existing limitations.

- *Evaluation metrics for code summarization tasks:* Current practices predominantly utilize text coherence metrics like BLEU-4 [64] and Rouge-L [66], which are originally designed for text translation tasks. However, these may not be entirely appropriate for binary code summarization. Reverse engineers can usually understand the specific design of a function through certain key terms, and text fluency is not that important at this time. It may be beneficial to develop a new metric to better capture the essence of binary summaries.
- *Binary code obfuscation:* Our evaluation dataset does not consider any form of binary code obfuscation, such as encryption or compiler-based obfuscation [67]. In fact, obfuscation can significantly change the form of pseudo code, making it challenging for models to accurately understand and interpret the code. This may affect the applicability of LLMs in real-world scenarios where confusion is prevalent.

VI. Conclusion

In this paper, we selected two representative tasks: (1) function name recovery, and (2) binary code summarization,

and designed an automated method to construct a benchmark to comprehensively evaluate the capabilities of Large Language Models (LLMs) to understand binary code.

Our findings indicate that LLMs demonstrate promising capabilities in advancing automated binary code understaning, and we call for more new research to focus on this important domain of software engineering to further enhance the capabilities of LLMs to play a more critical role in the complex task of binary code analysis.

Acknowledgement

We thank the reviewers for their insightful comments and suggestions. This work was supported in part by the Natural Science Foundation of China under Grant U20B2047, 62072421, 62002334, 62102386 and 62121002, and by Open Fund of Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation under Grant CSSAE-2021-007.

References

[1] G. Canfora, M. Di Penta, and L. Cerulo, "Achievements and challenges in software reverse engineering," *Commun. ACM*, vol. 54, no. 4, p. 142–151, apr 2011. [Online]. Available: https://doi.org/10.1145/1924421.1924451

[2] J. T. Giffin, S. Jha, and B. P. Miller, "Efficient context-sensitive intrusion detection," in *Network and Distributed System Security Symposium*, 2004.

[3] S. Alrabaee, M. Debbabi, and L. Wang, "A survey of binary code fingerprinting approaches: Taxonomy, methodologies, and features," *ACM Comput. Surv.*, vol. 55, no. 1, jan 2022. [Online]. Available: https://doi.org/10.1145/3486860

[4] Z. Zhang, W. You, G. Tao, Y. Aafer, X. Liu, and X. Zhang, "Stochfuzz: Sound and cost-effective fuzzing of stripped binaries by incremental and stochastic rewriting," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 659–676.

[5] X. Meng and B. P. Miller, "Binary code is not easy," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ser. ISSTA 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 24–35. [Online]. Available: https://doi.org/10.1145/2931037.2931047

[6] J. Patrick-Evans, L. Cavallaro, and J. Kinder, "Probabilistic naming of functions in stripped binaries," in *Proceedings of the 36th Annual Computer Security Applications Conference*, ser. ACSAC '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 373–385. [Online]. Available: https://doi.org/10.1145/3427228.3427265

[7] Hex-RaysSA, ""ida pro"," https://www.hex-rays.com/products/ida, 2024.

[8] NationalSecurityAgency, ""ghidra"," https://github.com/NationalSecurityAgency/ghidra, 2024.

[9] Vector35, ""binary ninja"," https://binary.ninja/, 2024.

[10] E. M. Gellenbeck and C. R. Cook, "An investigation of procedure and variable names as beacons during program comprehension," USA, Tech. Rep., 1991.

[11] H. Gao, S. Cheng, Y. Xue, and W. Zhang, "A lightweight framework for function name reassignment based on large-scale stripped binaries," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 607–619. [Online]. Available: https://doi.org/10.1145/3460319.3464804

[12] Y. David, U. Alon, and E. Yahav, "Neural reverse engineering of stripped binaries using augmented control flow graphs," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, nov 2020. [Online]. Available: https://doi.org/10.1145/3428293

[13] X. Jin, K. Pei, J. Y. Won, and Z. Lin, "Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1631–1645. [Online]. Available: https://doi.org/10.1145/3548606.3560612

[14] G. Chen, H. Gao, J. Zhang, Y. He, S. Cheng, and W. Zhang, "Investigating neural-based function name reassignment from the perspective of binary code representation," in *2023 20th Annual International Conference on Privacy, Security and Trust (PST)*, 2023, pp. 1–11.

[15] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock, and K. Vijay-Shanker, "Towards automatically generating summary comments for java methods," in *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 43–52. [Online]. Available: https://doi.org/10.1145/1858996.1859006

[16] A. Al-Kaswan, T. Ahmed, M. Izadi, A. A. Sawant, P. Devanbu, and A. van Deursen, "Extending source code pre-trained language models to summarise decompiled binaries," in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2023, pp. 260–271.

[17] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, nov 2021, pp. 8696–8708. [Online]. Available: https://aclanthology.org/2021.emnlp-main.685

[18] J. Xiong, G. Chen, K. Chen, H. Gao, S. Cheng, and W. Zhang, "Hext5: Unified pre-training for stripped binary code information inference," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 774–786.

[19] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, and Y. B. et.al., "Llama 2: Open foundation and fine-tuned chat models," 2023.

[20] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, and C. Z. et.al., "Training language models to follow instructions with human feedback," 2022.

[21] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, and J. L. et.al., "Code llama: Open foundation models for code," 2023.

[22] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," 2023.

[23] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, and S. Shah, "How effective are neural networks for fixing security vulnerabilities," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1282–1294. [Online]. Available: https://doi.org/10.1145/3597926.3598135

[24] Y. Zhang, W. Song, Z. Ji, Danfeng, Yao, and N. Meng, "How well does llm generate security tests?" 2023.

[25] Y. David, U. Alon, and E. Yahav, "Neural reverse engineering of stripped binaries using augmented control flow graphs," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, nov 2020. [Online]. Available: https://doi.org/10.1145/3428293

[26] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," 2023.

[27] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "Deepseek-coder: When the large language model meets programming – the rise of code intelligence," 2024.

[28] A. Zeng, X. Liu, Z. Du, Z. Wang, H. Lai, M. Ding, and Z. Y. et.al., "Glm-130b: An open bilingual pre-trained model," 2023.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," 2023.

[30] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024.

[31] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," 2021.

[32] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," 2023.

[33] Y. Zhang, "Leveraging artificial intelligence on binary code comprehension," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3551349.3559564

[34] FFmpeg, 2024. [Online]. Available: https://github.com/FFmpeg/FFmpeg

[35] Redis, 2024. [Online]. Available: https://github.com/redis/redis
[36] Curl, 2024. [Online]. Available: https://github.com/curl/curl
[37] Masscan, 2024. [Online]. Available: https://github.com/robertdavidgraham/masscan
[38] Llama2.c, 2024. [Online]. Available: https://github.com/karpathy/llama2.c
[39] Whisper.cpp, 2024. [Online]. Available: https://github.com/ggerganov/whisper.cpp
[40] OpenSSL, 2024. [Online]. Available: https://github.com/openssl/openssl
[41] zstd, 2024. [Online]. Available: https://github.com/facebook/zstd
[42] ImageMagick, 2024. [Online]. Available: https://github.com/ImageMagick/ImageMagick
[43] Libvips, 2024. [Online]. Available: https://github.com/libvips/libvips
[44] Libexpat, 2024. [Online]. Available: https://github.com/libexpat/libexpat
[45] Ultrajson, 2024. [Online]. Available: https://github.com/ultrajson/ultrajson
[46] I. U. International, "Dwarf debugging information format version 4," https://dwarfstd.org/doc/DWARF4.pdf, 2010.
[47] J. I. Maletic and M. L. Collard, "Exploration, analysis, and manipulation of source code using srcml," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 2, 2015, pp. 951–952.
[48] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, no. 1, p. 1418, 2024. [Online]. Available: https://doi.org/10.1038/s41467-024-45563-x
[49] D. Bzdok, A. Thieme, O. Levkovskyy, P. Wren, T. Ray, and S. Reddy, "Data science opportunities of large language models for neuroscience and biomedicine," *Neuron*, vol. 112, no. 5, pp. 698–717, Mar 2024. [Online]. Available: https://doi.org/10.1016/j.neuron.2024.01.016
[50] Z. Tan, A. Beigi, S. Wang, R. Guo, A. Bhattacharjee, B. Jiang, M. Karami, J. Li, L. Cheng, and H. Liu, "Large language models for data annotation: A survey," 2024.
[51] HuggingFace, 2024. [Online]. Available: https://huggingface.co/
[52] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in large language models: a comprehensive review," 2023.
[53] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better zero-shot reasoning with role-play prompting," 2024.
[54] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," 2020.
[55] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W. tau Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," 2023.
[56] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jtrans: jump-aware transformer for binary code similarity detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 1–13. [Online]. Available: https://doi.org/10.1145/3533767.3534367
[57] X. Li, Y. Qu, and H. Yin, "Palmtree: Learning an assembly language model for instruction embedding," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 3236–3251. [Online]. Available: https://doi.org/10.1145/3460120.3484587
[58] D. Kim, E. Kim, S. K. Cha, S. Son, and Y. Kim, "Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1661–1682, 2023.
[59] PyTorch, 2024. [Online]. Available: https://pytorch.org/
[60] DeepSpeed, 2024. [Online]. Available: https://www.deepspeed.ai/
[61] Transformers, 2024. [Online]. Available: https://huggingface.co/
[62] Y. Zheng, R. Zhang, J. Zhang, Y. Ye, Z. Luo, and Y. Ma, "Llamafactory: Unified efficient fine-tuning of 100+ language models," 2024.
[63] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021.
[64] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, 2002, pp. 311–318. [Online]. Available: https://aclanthology.org/P02-1040
[65] A. Lavie and M. J. Denkowski, "The meteor metric for automatic evaluation of machine translation," *Machine Translation*, vol. 23, no. 2–3, p. 105–115, sep 2009. [Online]. Available: https://doi.org/10.1007/s10590-009-9059-4
[66] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013
[67] P. Junod, J. Rinaldini, J. Wehrli, and J. Michielin, "Obfuscator-llvm – software protection for the masses," in *Proceedings of the 2015 IEEE/ACM 1st International Workshop on Software Protection*, ser. SPRO '15. USA: IEEE Computer Society, 2015, p. 3–9. [Online]. Available: https://doi.org/10.1109/SPRO.2015.10